



Programming Domains

CS3204N Programming Languages

PROGRAMMING DOMAINS

PROGRAMMING DOMAINS

refers to the specific category or field for which a software application is designed.

SCIENTIFIC APPLICATION

This is where computing began. These applications focus on heavy mathematical calculations.

Origins: Late 1940s – Early 1950s.

Key Requirements:

- Massive floating-point arithmetic.

Primary Languages:

- **Fortran:** Still used today for its unmatched efficiency in math.
- **ALGOL 60:** Influential ancestor of many modern languages.

SCIENTIFIC APPLICATION

This is where computing began. These applications focus on heavy mathematical calculations.

Origins: Late 1940s – Early 1950s.

Key Requirements:

- Massive floating-point arithmetic.

Primary Languages:

- **Fortran:** Still used today for its unmatched efficiency in math.
- **ALGOL 60:** Influential ancestor of many modern languages.

BUSINESS APPLICATION

Business software is all about record-keeping and reporting. Think of systems that manage payroll, inventory, or banking transactions.

Origins: 1950s.

Key Requirements:

- Production of elaborate, professional reports.
- Precise decimal arithmetic (crucial for finance).
- Handling character data (names, addresses).

Primary Language:

- **COBOL:** The undisputed leader since 1960. Still runs much of the world's financial infrastructure today.

ARTIFICIAL INTELLIGENCE

AI focuses on symbolic computation rather than just numbers. Instead of adding $1 + 1$, an AI language might be used to define relationships, like "A human is a mammal."

Characteristics:

- Manipulates names/symbols instead of just numbers.
- Uses **linked lists** rather than arrays.
- Requires high flexibility (dynamic code execution).

Primary Languages:

- **Lisp (1959):** The pioneer of functional AI programming.
- **Prolog:** Logic-based approach (70s).
- **Python:** The modern systems language of choice for AI.

WEB SOFTWARE

The modern "eclectic" domain. Web software isn't just one language; it's a stack of different tools working together to make a webpage interactive.

A "Hybrid" Domain: The Web uses an eclectic collection of technologies.

Usual Languages:

- **Markup:** HTML (Defines structure, not a programming language).
- **Scripting:** JavaScript and PHP (Embedded in HTML for dynamic content).
- **General Purpose:** Java (Used for complex web logic).



Language Evaluation Criteria

CS3204N Programming Languages

LANGUAGE EVALUATION CRITERIA

LANGUAGE EVALUATION CRITERIA

Framework for assessing programming languages

Focus on impact on:

- Software development
- Maintenance
- Reliability
- Cost

Primary Evaluation Criteria

- Readability
- Writability
- Reliability
- Cost

*Note: These criteria do **not** have equal importance
Readability & Writability are the most influential*

Characteristic	CRITERIA		
	READABILITY	WRITABILITY	RELIABILITY
Simplicity	•	•	•
Orthogonality	•	•	•
Data types	•	•	•
Syntax design	•	•	•
Support for abstraction		•	•
Expressivity		•	•
Type checking			•
Exception handling			•
Restricted aliasing			•

Readability

Ease with which programs can be read and understood

Importance:

- Maintenance dominates software lifecycle cost
- Shift from **machine-oriented** to **human-oriented** languages

Readability improves:

- Debugging
- Maintenance
- Long-term reliability

READABILITY FACTORS

READABILITY FACTORS

- Overall Simplicity
- Orthogonality
- Data Types
- Syntax Design

OVERALL SIMPLICITY

The readability of a programming language is heavily influenced by its overall simplicity. While a language should be powerful enough to express complex ideas, an excess of features, multiple ways to perform the same task, or confusing symbol meanings can make code difficult to learn, maintain, and share between developers.

Fewer basic constructs → easier learning

Potential Problems:

- Feature multiplicity (many ways to do the same thing)
- Excessive operator overloading

ORTHOGONALITY

Orthogonality is a design principle where a small set of primitive building blocks can be combined in consistent, predictable ways. In an orthogonal language, "rules have no exceptions"—if a feature works in one context, it should work in all others. This makes a language easier to learn and write, though extreme orthogonality can lead to overly complex and unreadable code.

Small set of primitives + consistent rules

Fewer exceptions → higher readability

DATA TYPES

The ability of a language to define clear, descriptive data types and structures directly impacts how easily a program can be read and understood. When a language provides specialized types (like Booleans) instead of forcing programmers to use generic numbers to represent logical states, the code becomes "self-documenting" and less prone to human error.

Clear, well-defined data types improve meaning

Example:

timeout = 1 (ambiguous)

timeout = true (clear)

SYNTAX DESIGN

Syntax refers to the physical "form" of a programming language. A well-designed syntax ensures that the code's appearance reflects its actual meaning (semantics). When syntax is ambiguous—such as using the same symbol to end different types of loops or allowing special words to be used as variable names—it increases the "mental tax" on the programmer and leads to errors.

- **Clear special words** *e.g. (end if, end loop)*
- **Syntax should reflect meaning**
- **Avoid context-dependent semantics**

Writability

Ease of using a language to create programs for a specific domain.

Closely related to readability

Must be evaluated **within the problem domain**

Example:

- Visual Basic → GUI development
- C → systems programming

WRITABILITY FACTORS

Writability and Orthogonality

Writability measures how easily a language can be used to create programs for a chosen domain. While having many features might seem like an advantage, true writability comes from a small, consistent set of rules (orthogonality). However, there is a "tipping point" where too much freedom prevents the compiler from catching logical errors, making the language dangerous to use.

- Fewer constructs → fewer errors
- Consistent rules improve ease of writing
- Too much orthogonality can hide errors

Expressivity

Expressivity refers to how powerfully and conveniently a language allows a programmer to specify computations. A highly expressive language provides "shortcuts" or specialized structures that reduce the amount of code needed to perform common tasks, making the language more writable and elegant.

Examples:

- `count++` vs `count = count + 1`
- Short-circuit Boolean operators
- for loops for counting

Reliability

Ability of a program to perform correctly under all conditions

Influencing Factors:

- Type checking
- Exception handling
- Aliasing control
- Readability & writability

RELIABILITY FACTORS

Type Checking

Type checking is the process of verifying that a program handles data types consistently (e.g., not trying to add a "word" to a "number"). By catching these "type errors" early—ideally during compilation—a language prevents nonsensical data processing and system crashes, making the software significantly more reliable.

- Compile-time checking preferred
- Prevents runtime type errors
- Example: Java vs early C

Exception Handling

Exception handling is a language's ability to intercept run-time errors—such as dividing by zero or failing to open a file—and take corrective action instead of simply crashing. By providing a structured way to manage "unusual conditions," the language ensures that a program can remain functional even when something goes wrong.

- Graceful recovery from runtime errors
- Supported in Java, C++, Ada, C#
- Limited or absent in C

Aliasing

Aliasing occurs when a single memory location can be accessed through two or more different names (or variables) in a program. While often used for flexibility, aliasing is considered a dangerous feature because it allows "hidden" side effects: changing one variable can unexpectedly change another, making the program's behavior difficult to predict and debug.

- Can cause unintended side effects
- Makes programs harder to reason about
- Restricted in reliable language designs
- Common source of bugs in pointer-based languages

Readability & Writability

Reliability is not an isolated metric; it is the direct result of how readable and writable a language is. If a language makes it difficult to express logic naturally, programmers are forced to use "workarounds" that increase the likelihood of bugs. A language that is easy to read and write is fundamentally more likely to produce code that is correct and maintainable.

- Can cause unintended side effects
- Makes programs harder to reason about
- Restricted in reliable language designs
- Common source of bugs in pointer-based languages

KEY POINTS

- **Natural Expression:** When a language supports "natural" ways to express an algorithm, the code is more likely to handle all possible situations correctly.
- **The Danger of "Unnatural" Approaches:** If a language lacks the right tools, programmers use "hacks" or complex workarounds. These are prone to errors because they don't directly map to the problem being solved.
- **Readability and the Life Cycle:** * **Writing Phase:** If you can't read what you've just written, you can't verify its logic.
- **Maintenance Phase:** Programs are modified over time. If the code is difficult to read, future changes are likely to introduce new, accidental bugs.
- **The Reliability Cycle:** High Readability + High Writability = Code that is easier to verify, easier to fix, and ultimately more Reliable.

Cost

The total cost of a programming language isn't just the price of a license; it is a complex sum of human time, machine resources, and potential risks. From initial training to long-term maintenance, the design of a language—specifically its readability and writability—directly dictates the financial and operational success of a software project.

GET 1/2 SHEET OF PAPER

1. The specific category or field for which a software application is designed.

2. The programming domain where computing began, characterized by heavy mathematical calculations and massive floating-point arithmetic.

3. The primary programming language still widely used today for scientific applications due to its efficiency in mathematical computations.

4. The programming domain that focuses on record-keeping, reporting, payroll, inventory, and banking transactions.

5. The programming language that has dominated business applications since the 1960s and still runs much of the world's financial systems.

6. The programming domain that emphasizes symbolic computation, relationships, and rule-based logic instead of numerical calculations.

7. The AI programming language known for its logic-based approach and use of rules and facts.

8. The modern programming domain described as “eclectic” because it uses a combination of multiple technologies and languages.

9. The evaluation framework used to assess programming languages based on their impact on software development, maintenance, reliability, and cost.

10. The primary evaluation criterion that refers to the ease with which a program can be read and understood.

11. The readability factor that promotes consistent rules and minimal exceptions in a programming language.

12. The readability factor that concerns the clarity and meaningfulness of data representations used in a program.

13. The evaluation criterion that measures how easily a language can be used to create programs for a specific problem domain.

14. The reliability factor that ensures data types are used consistently and helps prevent runtime type errors.

15. The condition in programming where a single memory location can be accessed using two or more different variable names, potentially causing hidden side effects.

ANSWER KEY